AUCTION MANAGEMENT PLATFORM

A Mini Project Report Submitted in the partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

In

DEPARTMENT OF COMPUTER SCIENCE ENGINNERING

By

Masuma Fathema – 2420030400

Avinash Yarukala – 2420030568

Under the Esteemed Guidance of

Mrs. E. Uma Shankari



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING



DECLARATION

The Mini Project Report entitled "Auction Management Platform: A Full-Stack Web-Based System for Transparent Real-Time Bidding" is a record of bonafide work carried out by Masuma Fathema (2420030400) and Avinash Yarukala (2420030568), submitted in partial fulfilment of the requirements for the award of B.Tech in Computer Science and Engineering at K L (Deemed to be) University. The work has been completed under the guidance of Mrs. E. Uma Shankari. The results and descriptions in this report relate to the project work undertaken by the team, and external sources used have been acknowledged in the References section.

Masuma Fathema – 2420030400

Avinash Yarukala – 2420030568

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING



CERTIFICATE

This is to certify that the mini project report entitled “Auction Management Platform: A Full-Stack Web-Based System for Transparent Real-Time Bidding” is a bonafide work carried out and submitted by Masuma Fathema (2420030400) and Avinash Yarukala (2420030568), students of the Department of Computer Science Engineering, K L (Deemed to be) University, in partial fulfilment of the requirements for the award of the degree of Bachelor of Technology during the academic year 2026–2027, under the guidance of Mrs. E. Uma Shankari.

Signature of the Course Co-ordinator **Signature of HOD**

ACKNOWLEDGEMENT

We express our sincere gratitude to our project guide, Mrs. E. Uma Shankari, Department of Computer Science Engineering, for her valuable guidance, encouragement and suggestions throughout the development of the Auction Management Platform.

We thank the Department of Computer Science Engineering and KLH CSE Bowrampet Campus for providing the academic environment and facilities required to undertake this project. The guidance and feedback received during the project reviews helped us refine the problem definition, architecture and implementation plan.

We also acknowledge the support of the Head of the Department, faculty members and technical staff for their assistance and for providing the resources needed to complete the project work.

Finally, we thank our families, classmates and everyone who supported us directly or indirectly during the planning, development, testing and documentation of this mini project.

Masuma Fathema – 2420030400

Avinash Yarukala – 2420030568

TABLE OF CONTENTS

S.No.	Contents	Page no.
1.	Abstract	6
2.	Introduction	8
3.	Literature survey	9
4.	Problem Statement and Objectives	10
5.	Hardware and Software requirements	11
6.	Implementation	12
7.	Results & Discussion	13
8.	Conclusion & Future Scope	14
9.	References	15

1. ABSTRACT

The Auction Management Platform is a full-stack web application designed to make online auctions more accessible, transparent and easier to manage. Traditional auction processes may depend on physical participation or manual coordination, which can delay bid tracking, communication and auction settlement. The proposed platform brings the main auction activities into one centralized system with three roles: buyer, seller and administrator. Sellers can list products and schedule auctions, while buyers can browse items and submit bids subject to server-side validation and minimum-increment rules. WebSocket communication is used to publish accepted bid updates to connected clients without requiring a page refresh. The system also includes proxy bidding, automatic auction closure, winner identification, payment-status tracking, digital invoice generation, notifications and administrative monitoring. The frontend uses React.js, HTML5, CSS3 and JavaScript. The backend uses Java Spring Boot and REST APIs, with Spring Security and JSON Web Tokens for authentication and role-based authorization. MySQL is the intended relational database, with an H2 demo profile where configured for local evaluation. The project follows a modular development approach so that individual functions can be integrated and tested systematically. The platform is intended to provide a consistent record of listings, bids and auction outcomes.

Team Name: Auction Management Platform Project Team

Team Members:

1. Masuma Fathema – 2420030400 – Frontend, UI integration, testing and documentation.

2. Avinash Yarukala – 2420030568 – Backend, database integration, bidding and system integration.

Project Guide: Mrs. E. Uma Shankari – Department of Computer Science Engineering.

Institution: KLH CSE Bowrampet Campus, K L (Deemed to be) University.

Project Domain: Software Engineering / Web Application Development.

Technology Focus: React.js, Spring Boot, MySQL, JWT and WebSocket.

2. INTRODUCTION

Online auctions allow buyers and sellers to exchange products without being physically present at the same location. A successful auction application must coordinate user accounts, product listings, auction schedules, bid validation, auction closure and post-auction transactions. When these activities are managed manually or through separate systems, users may face delayed updates, inconsistent records and uncertainty about the final result. A web-based auction platform can bring these activities together and make the process easier to access.

The Auction Management Platform is designed as a centralized application for buyers, sellers and administrators. Sellers can publish items and configure auction start and end times. Buyers can browse active listings and place bids. The backend checks each bid against the auction state and minimum increment before saving it. Accepted bids are shared with connected users through WebSocket communication. At the end of an auction, the system records the highest eligible bid as the winner and supports payment-status tracking and invoice generation. The remainder of this report explains the related work, project objectives, requirements, architecture, implementation, results and possible future enhancements.

3. LITERATURE SURVEY

This section reviews research papers and existing auction platforms related to online bidding, proxy bidding, security and auction management.

1. Rumpe and Wimmel (2014), "A Framework for Realtime Online Auctions": This work discusses the architecture and technical requirements of real-time online auction systems. It emphasizes structured design, reuse and iterative development. These ideas support the modular architecture selected for this project. The proposed platform extends the workflow with role-based access, live bid updates and payment-related modules. URL: https://arxiv.org/abs/1409.6592

2. Cai, Chen and Gong (2012), "Design of Online Auctions: Proxy versus Non-Proxy Settings": This study compares proxy and non-proxy auction settings and examines bidder behavior and seller revenue. It provides a research basis for automated bidding mechanisms. The proposed project includes proxy bidding with a bidder-defined maximum limit and configured bid increments. The actual behavior of the implemented module must be verified through functional tests. URL: https://doi.org/10.1016/j.dss.2011.09.005

3. Borkar, Dave and Shyamasundar (2005), "SeTiA: Secure Multi Auction System": The SeTiA research addresses auction mechanisms, confidentiality, security and system architecture. It highlights the importance of secure transactions and controlled auction closure. These concerns inform the use of Spring Security, JWT and server-side bid validation in this project. The current platform uses a modern React and Spring Boot architecture. URL: https://doi.org/10.1515/jisys.2005.14.1.45

4. eBay – Online Marketplace and Auction Listings: eBay provides an established marketplace with product listings and auction-style buying options. Its public interface demonstrates catalogue browsing, item descriptions, bid information and post-auction purchase workflows. These features provide a reference for designing the product catalogue and auction detail screens. The academic project implements a simplified independent auction workflow. URL: https://www.ebay.com/

5. Catawiki – Online Auctions: Catawiki provides curated auctions across several product categories. Its catalogue demonstrates category-based discovery, item presentation and time-bound auction participation. These concepts are relevant to the listing and scheduling modules of the proposed platform. The project uses its own role-based seller and buyer workflows. URL: https://www.catawiki.com/

6. LiveAuctioneers – Online Auctions and Live Bidding: LiveAuctioneers supports online auction discovery and live bidding for participating auction houses. It is a useful reference for presenting changing bid information to participants. The proposed system uses WebSocket messaging to distribute accepted bid updates to connected clients. The project focuses on implementing this behavior in a full-stack academic application. URL: https://www.liveauctioneers.com/

7. Invaluable – Auction Catalogues and Results: Invaluable provides auction catalogues and auction-related information from participating auction houses. Its public pages illustrate item discovery, catalogue organization and auction result presentation. These ideas are relevant to the

search, product detail and completed-auction views in this project. The proposed platform combines these views with seller and administrator dashboards. URL: https://www.invaluable.com/

8. Heritage Auctions – Online Auctions: Heritage Auctions provides online catalogues and bidding experiences across specialist categories. Its item pages illustrate the importance of detailed descriptions, images and auction schedules. These features informed the proposed product listing and auction detail screens. The project provides a smaller integrated implementation for academic demonstration. URL: https://www.ha.com/

4. Problem Statement and Objectives

Problem Statement:

To develop a secure, centralized online auction platform that supports transparent real-time bidding and coordinated auction management.

Problem Description:

Traditional auctions may involve geographical limitations, manual coordination, delayed bid tracking and disconnected payment processes. The proposed platform integrates listings, bidding, auction closure and post-auction records.

Objectives:

The purpose of this project is to simplify and centralize the online auction lifecycle for buyers, sellers and administrators.

The project aims to provide secure role-based access, product listing, auction scheduling, minimum-increment bid validation, real-time bid updates, proxy bidding, automatic winner identification, payment-status tracking, invoice generation, notifications and administrative monitoring.

1. Develop a full-stack web application using React.js, Spring Boot and a relational database.
2. Provide buyer, seller and administrator roles with authenticated access.
3. Allow sellers to list products and configure auction schedules and minimum increments.
4. Allow buyers to browse active auctions and submit eligible bids.
5. Validate bids on the server and broadcast accepted bids through WebSockets.
6. Support proxy bidding with a maximum limit and configured increment rules.
7. Close auctions automatically and identify the highest eligible bidder.
8. Maintain payment and invoice records and provide notification functionality.
9. Provide an admin dashboard for user and auction management.

5. HARDWARE AND SOFTWARE REQUIREMENTS

Hardware Requirements:

- • Processor: Intel Core i3 or equivalent and above.
- • RAM: 8 GB minimum; 16 GB recommended for running the IDE, browser and servers together.
- • Storage: At least 5 GB free space for source code, dependencies and local database files.

Software Requirements:

- • Operating System: Windows 10/11 or compatible operating system.
- • Frontend: React.js, Vite, HTML5, CSS3 and JavaScript.
- • Backend: Java and Spring Boot with REST API support.
- • Security: Spring Security 6 and JWT bearer-token authentication.
- • Real-time communication: STOMP over SockJS / WebSocket.

• Database: MySQL; H2 demo profile where configured.

• API communication: Axios.

• Build tools: Maven and npm.

• Development tools: Visual Studio Code / IntelliJ IDEA, Git and GitHub.

• Browser: Recent version of Google Chrome, Microsoft Edge or Firefox.

6. IMPLEMENTATION

1. Frontend: React.js with Vite, HTML5, CSS3 and JavaScript. The frontend contains the landing page, login page, product catalogue, auction details, live bidding interface and separate buyer, seller and admin dashboards.
2. Backend: Java Spring Boot with REST controllers, service classes, repositories and JPA entities. Spring Security and JWT provide stateless authentication and role-based authorization. MySQL is used for relational persistence, with an H2 demo profile where configured.
3. API Integration: Axios is used for REST API requests. STOMP over SockJS provides WebSocket communication through /ws-auction, with auction updates published to /topic/auctions/{id}.

1. Frontend (React.js, HTML, CSS and JavaScript)

A. src/main.jsx – React entry point

```
import React from "react";
import ReactDOM from "react-dom/client";
import App from "./App";
import "./index.css";

ReactDOM.createRoot(
  document.getElementById("root")
).render(
  <React.StrictMode>
    <App />
  </React.StrictMode>
);
```

B. src/App.jsx – Main application component

```
import React from "react";
```

```
import { BrowserRouter, Routes, Route } from "react-router-dom";

import Login from "./pages/Login";
import BuyerDashboard from "./pages/BuyerDashboard";
import SellerDashboard from "./pages/SellerDashboard";
import AdminDashboard from "./pages/AdminDashboard";

function App() {
  return (
    <BrowserRouter>
      <Routes>
        <Route path="/" element={<Login />} />
        <Route path="/buyer" element={<BuyerDashboard />} />
        <Route path="/seller" element={<SellerDashboard />} />
        <Route path="/admin" element={<AdminDashboard />} />
      </Routes>
    </BrowserRouter>
  );
}

export default App;
```

C. src/index.css – Global styling

```
* {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
}
```

```
body {
  font-family: "Plus Jakarta Sans", sans-serif;
  background: #f8f7f4;
  color: #172033;
}

button {
  cursor: pointer;
  border: none;
  border-radius: 6px;
  padding: 10px 18px;
  background: #d97706;
  color: white;
}

.container {
  width: 90%;
  margin: auto;
}

.card {
  background: white;
  padding: 20px;
  border-radius: 12px;
  box-shadow: 0 2px 10px rgba(0,0,0,0.08);
}

@media (max-width: 768px) {
```

```
  .container {
    width: 95%;
  }
}
```

D. src/App.css – Application layout

```
.navbar {
  display: flex;
  justify-content: space-between;
  align-items: center;
  padding: 20px 5%;
  background: #172033;
  color: white;
}

.dashboard {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  gap: 20px;
  padding: 30px;
}

.product-card {
  background: white;
  border-radius: 12px;
  padding: 15px;
  overflow: hidden;
}
```

```
.product-card img {
  width: 100%;
  height: 200px;
  object-fit: cover;
}

@media (max-width: 768px) {
  .dashboard {
    grid-template-columns: 1fr;
  }
}
```

E. ProductCard.jsx – Product display component

```
function ProductCard({ product }) {
  return (
    <div className="product-card">
      <img src={product.imageUrl} alt={product.title} />

      <h3>{product.title}</h3>
      <p>{product.description}</p>

      <h4>Current Bid: ₹{product.currentPrice}</h4>

      <button>View Auction</button>
    </div>
  );
}
```

```
export default ProductCard;
```

F. LiveBidding.jsx – Live bidding component

```
import { useState } from "react";
import axios from "axios";

function LiveBidding({ auctionId, token }) {
  const [amount, setAmount] = useState("");

  const placeBid = async () => {
    try {
      await axios.post(
        `/api/bids/${auctionId}`,
        { amount: Number(amount) },
        {
          headers: {
            Authorization: `Bearer ${token}`
          }
        }
      );

      alert("Bid placed successfully!");
    } catch (error) {
      alert("Bid placement failed");
    }
  };

  return (
```

```
    <div className="card">
      <h3>Place Your Bid</h3>

      <input
        type="number"
        value={amount}
        onChange={(e) => setAmount(e.target.value)}
        placeholder="Enter bid amount"
      />

      <button onClick={placeBid}>
        Submit Bid
      </button>
    </div>
  );
}

export default LiveBidding;
```

G. Dashboard components

BuyerDashboard.jsx

```
function BuyerDashboard() {
  return (
    <div className="container">
      <h1>Buyer Dashboard</h1>
      <p>Browse auctions and participate in live bidding.</p>
      <button>View Auctions</button>
      <button>My Bids</button>
```

```
      <button>My Invoices</button>
    </div>
  );
}

export default BuyerDashboard;
```

SellerDashboard.jsx

```
function SellerDashboard() {
  return (
    <div className="container">
      <h1>Seller Dashboard</h1>
      <p>Manage products and scheduled auctions.</p>
      <button>Add Product</button>
      <button>Create Auction</button>
      <button>View Bids</button>
    </div>
  );
}

export default SellerDashboard;
```

AdminDashboard.jsx

```
function AdminDashboard() {
  return (
    <div className="container">
      <h1>Admin Dashboard</h1>
      <p>Monitor platform activities.</p>
      <button>Manage Users</button>
      <button>Manage Auctions</button>
```

```
      <button>View Reports</button>
    </div>
  );
}

export default AdminDashboard;
```

2. Backend (Java / Spring Boot)

A. AuctionPlatformApplication.java – Main application class

```
package com.auction;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.scheduling.annotation.EnableScheduling;

@SpringBootApplication
@EnableScheduling
public class AuctionPlatformApplication {

    public static void main(String[] args) {
        SpringApplication.run(
            AuctionPlatformApplication.class, args
        );
    }
}
```

B. AuthController.java – Authentication controller

```
@RestController
@RequestMapping("/api/auth")
public class AuthController {

   @Autowired
   private AuthenticationManager authenticationManager;

   @Autowired
   private JwtService jwtService;

   @PostMapping("/login")
   public ResponseEntity<?> login(
         @RequestBody LoginRequest request) {

      Authentication auth =
         authenticationManager.authenticate(
            new UsernamePasswordAuthenticationToken(
               request.getUsername(),
               request.getPassword()
            )
         );

      String token =
         jwtService.generateToken(auth);

      return ResponseEntity.ok(token);
   }
```

```
}
```

C. SecurityConfig.java – Security configuration

```
@Configuration
@EnableWebSecurity
public class SecurityConfig {

   @Bean
   public SecurityFilterChain securityFilterChain(
         HttpSecurity http) throws Exception {

      http
         .csrf(csrf -> csrf.disable())
         .sessionManagement(session ->
            session.sessionCreationPolicy(
               SessionCreationPolicy.STATELESS
            )
         )
         .authorizeHttpRequests(auth -> auth
            .requestMatchers("/api/auth/**").permitAll()
            .requestMatchers("/api/admin/**")
               .hasRole("ADMIN")
            .requestMatchers("/api/seller/**")
               .hasRole("SELLER")
            .requestMatchers("/api/bids/**")
               .hasRole("BUYER")
            .anyRequest().authenticated()
         );
```

```
        return http.build();
    }
}
```

Note: A complete JWT filter and password encoder must also be configured.

D. ProductController.java – Product management

```
@RestController
@RequestMapping("/api/products")
public class ProductController {

    @Autowired
    private ProductRepository productRepository;

    @GetMapping
    public List<Product> getProducts() {
        return productRepository.findAll();
    }

    @PostMapping
    public Product addProduct(
            @RequestBody Product product) {

        return productRepository.save(product);
    }

    @GetMapping("/{id}")
    public Product getProduct(@PathVariable Long id) {
```

```
        return productRepository.findById(id)
            .orElseThrow(() ->
                new RuntimeException("Product not found")
            );
    }
}
```

E. AuctionController.java – Auction management

```
@RestController
@RequestMapping("/api/auctions")
public class AuctionController {

    @Autowired
    private AuctionRepository auctionRepository;

    @GetMapping
    public List<Auction> getAuctions() {
        return auctionRepository.findAll();
    }

    @PostMapping
    public Auction createAuction(
            @RequestBody Auction auction) {

        auction.setStatus("SCHEDULED");
        return auctionRepository.save(auction);
    }
```

```
    @GetMapping("/{id}")
    public Auction getAuction(@PathVariable Long id) {
        return auctionRepository.findById(id)
            .orElseThrow(() ->
                new RuntimeException("Auction not found")
            );
    }
}
```

F. BidController.java – Bidding operations

```
@RestController
@RequestMapping("/api/bids")
public class BidController {

    @Autowired
    private BidService bidService;

    @PostMapping("/{auctionId}")
    public ResponseEntity<?> placeBid(
            @PathVariable Long auctionId,
            @RequestBody BidRequest request) {

        Bid bid = bidService.placeBid(
            auctionId, request.getAmount()
        );

        return ResponseEntity.ok(bid);
    }
```

```
}
```

G. WebSocketConfig.java – Real-time communication

```
@Configuration
@EnableWebSocketMessageBroker
public class WebSocketConfig
      implements WebSocketMessageBrokerConfigurer {

   @Override
   public void configureMessageBroker(
         MessageBrokerRegistry config) {

      config.enableSimpleBroker("/topic");
      config.setApplicationDestinationPrefixes("/app");
   }

   @Override
   public void registerStompEndpoints(
         StompEndpointRegistry registry) {

      registry.addEndpoint("/ws-auction")
            .setAllowedOriginPatterns("*")
            .withSockJS();
   }
}
```

H. Repository classes

ProductRepository.java

```
@Repository
public interface ProductRepository
        extends JpaRepository<Product, Long> {
}
```

AuctionRepository.java

```
@Repository
public interface AuctionRepository
        extends JpaRepository<Auction, Long> {
}
```

BidRepository.java

```
@Repository
public interface BidRepository
        extends JpaRepository<Bid, Long> {

    List<Bid> findByAuctionIdOrderByAmountDesc(
        Long auctionId
    );
}
```

3. API Integration

A. api.js – Axios configuration

```
import axios from "axios";

const api = axios.create({
    baseURL: "http://localhost:8080/api"
});

api.interceptors.request.use((config) => {
```

```
    const token = localStorage.getItem("token");

    if (token) {
        config.headers.Authorization = `Bearer ${token}`;
    }

    return config;
});

export default api;
```

B. auctionService.js – Auction API integration

```
import api from "./api";

export const getAuctions = async () => {
    const response = await api.get("/auctions");
    return response.data;
};

export const createAuction = async (data) => {
    const response = await api.post("/auctions", data);
    return response.data;
};

export const placeBid = async (id, amount) => {
    const response = await api.post(
        `/bids/${id}`,
        { amount }
```

```
  );

  return response.data;
};
```

C. websocket.js – Live bid updates

```
import SockJS from "sockjs-client";
import { Client } from "@stomp/stompjs";

export function connectAuction(auctionId, onUpdate) {

  const client = new Client({
    webSocketFactory: () =>
      new SockJS("http://localhost:8080/ws-auction"),

    onConnect: () => {
      client.subscribe(
        `/topic/auctions/${auctionId}`,
        (message) => {
          onUpdate(JSON.parse(message.body));
        }
      );
    }
  });
  client.activate();
  return client;
}
```

6.1 Development Methodology: The project follows an Agile, module-based approach. Requirements and design are followed by user management, product and auction management, bidding, payment/notification integration, module integration and testing.

6.2 Authentication: Users are assigned ROLE_BUYER, ROLE_SELLER or ROLE_ADMIN. JWT tokens support stateless authentication, while backend role checks restrict protected operations.

6.3 Bidding Engine: The backend checks auction status, bidder permission and minimum bid increment before accepting a bid. Accepted bids are stored with bidder, auction, amount and timestamp. Concurrency control is used to protect simultaneous bid updates.

6.4 Proxy Bidding: A buyer can define a maximum bidding amount. The system can automatically place counter-bids according to configured increment rules without exceeding that maximum.

6.5 Auction Closure: A scheduled backend task checks auction end times, closes expired auctions, rejects further bids and records the highest eligible bidder as the winner.

6.6 Payment and Invoice: The system maintains payment status and generates invoice details for the completed auction. Demo payment flows and simulated transaction references are not real payment gateway transactions.

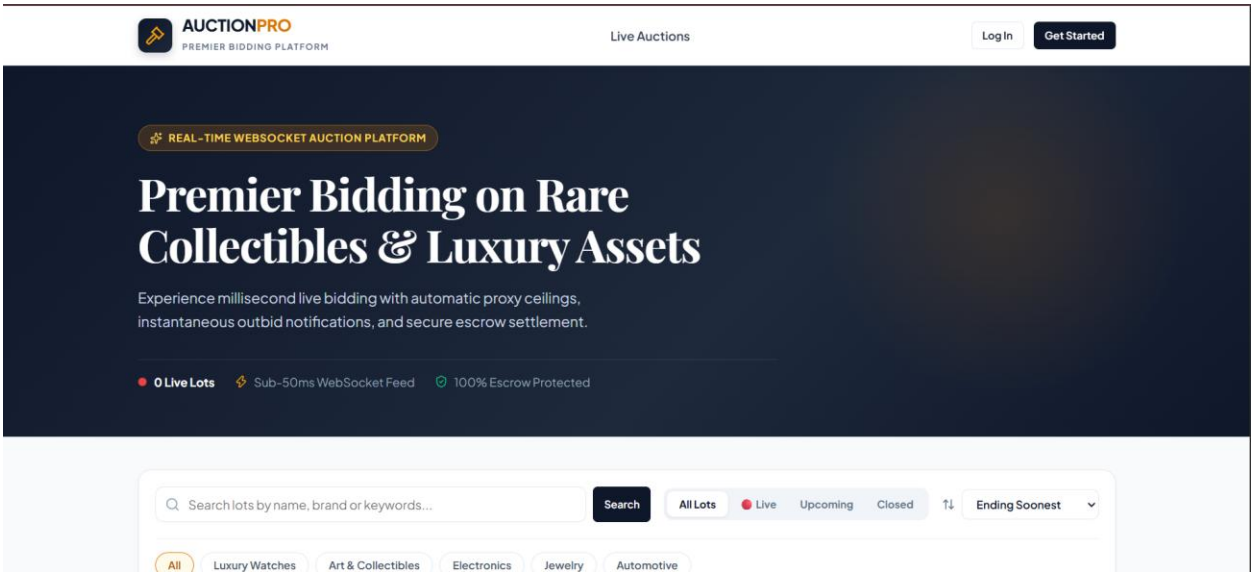
6.7 Notifications and Administration: In-app notifications track unread/read status. Email/SMS may be simulated through backend logs when external credentials are unavailable. Admin functions support monitoring and management of users and auctions.

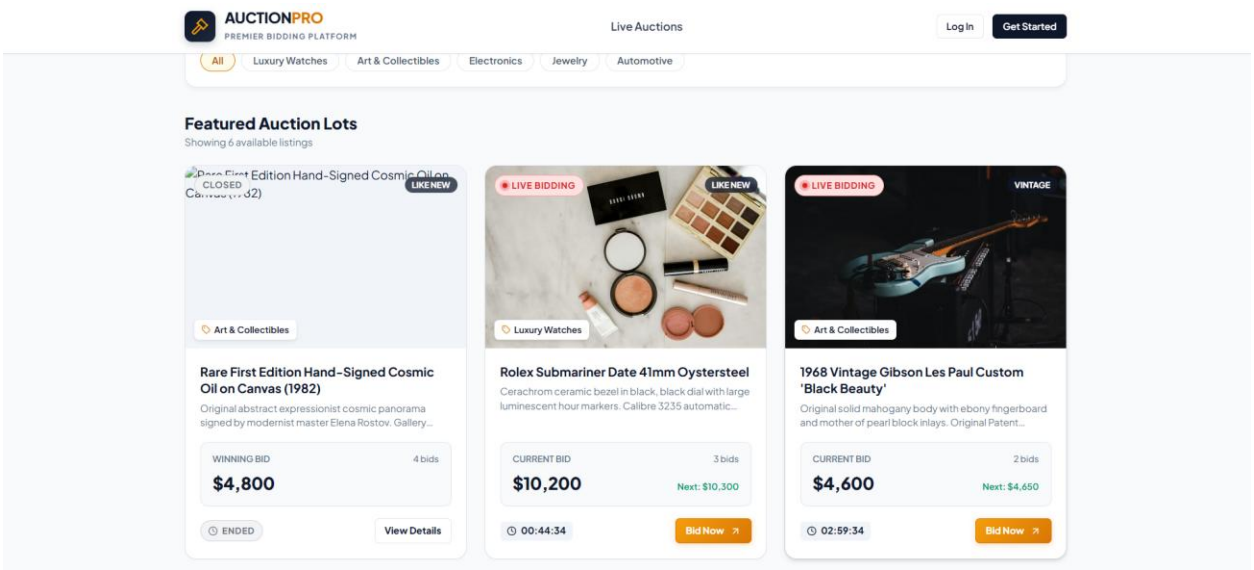
System Architecture: React.js Frontend ⇄ Axios / REST APIs ⇄ Spring Boot Controllers and Services ⇄ JPA / Hibernate ⇄ MySQL (or configured H2 demo database). WebSocket/STOMP carries accepted bid updates to subscribed auction clients. JWT and Spring Security protect role-specific operations.

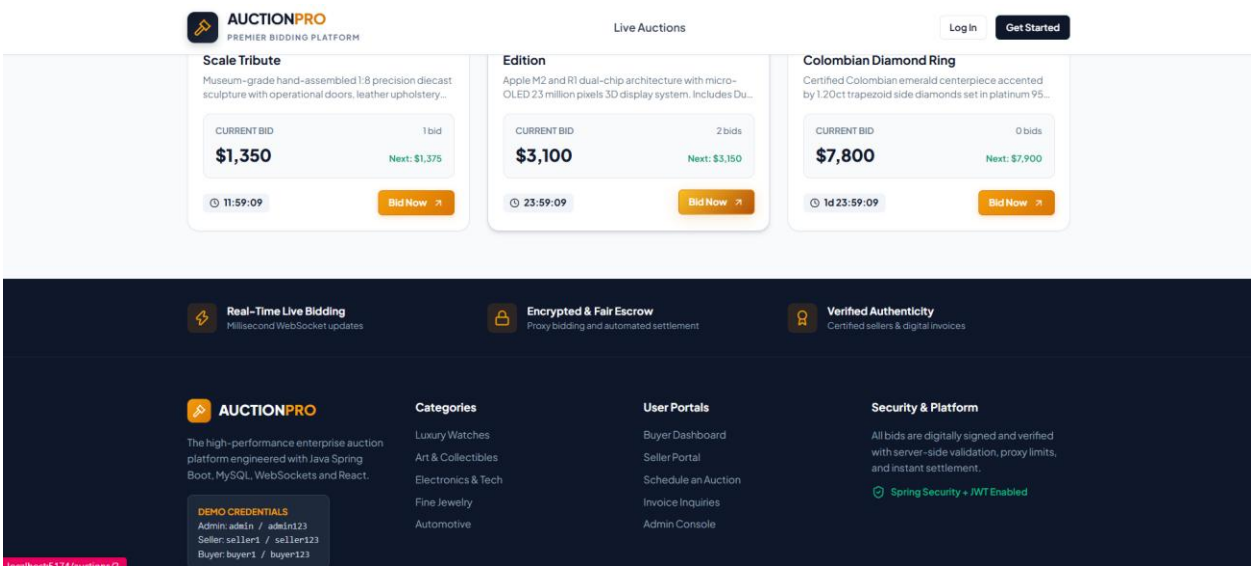
Auction Flow: Seller creates item → Seller schedules auction → Auction becomes active → Buyer submits bid → Backend validates user, auction status and minimum increment → Valid bid is stored → WebSocket broadcasts updated price → Auction end time reached → Auction closes → Winner and payment record are created → Invoice is generated.

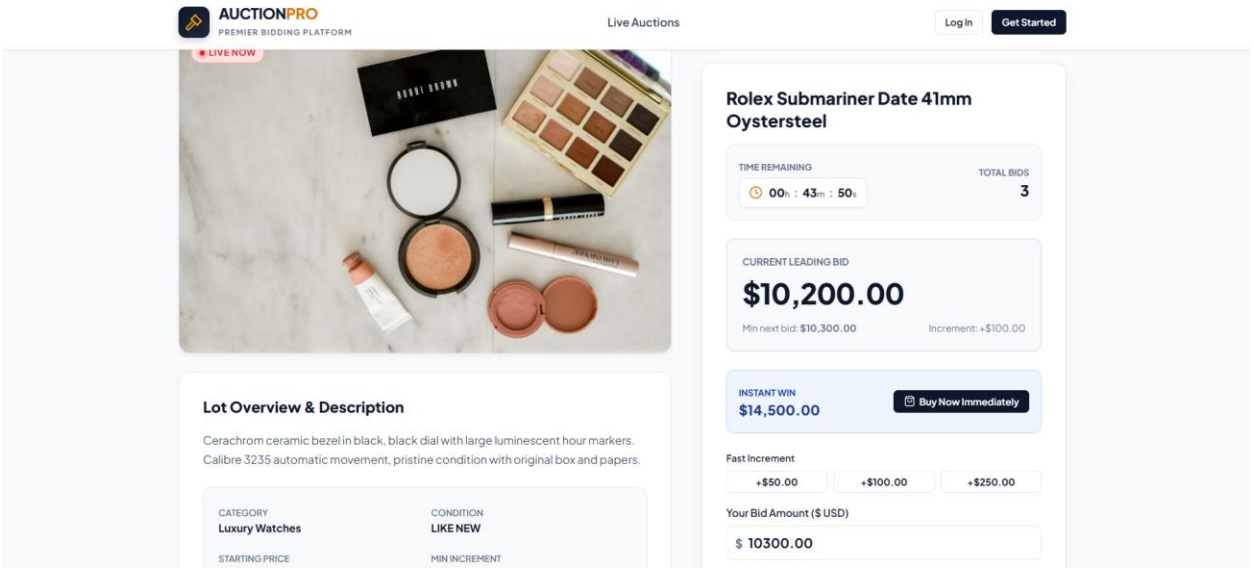
7. RESULTS & DISCUSSION

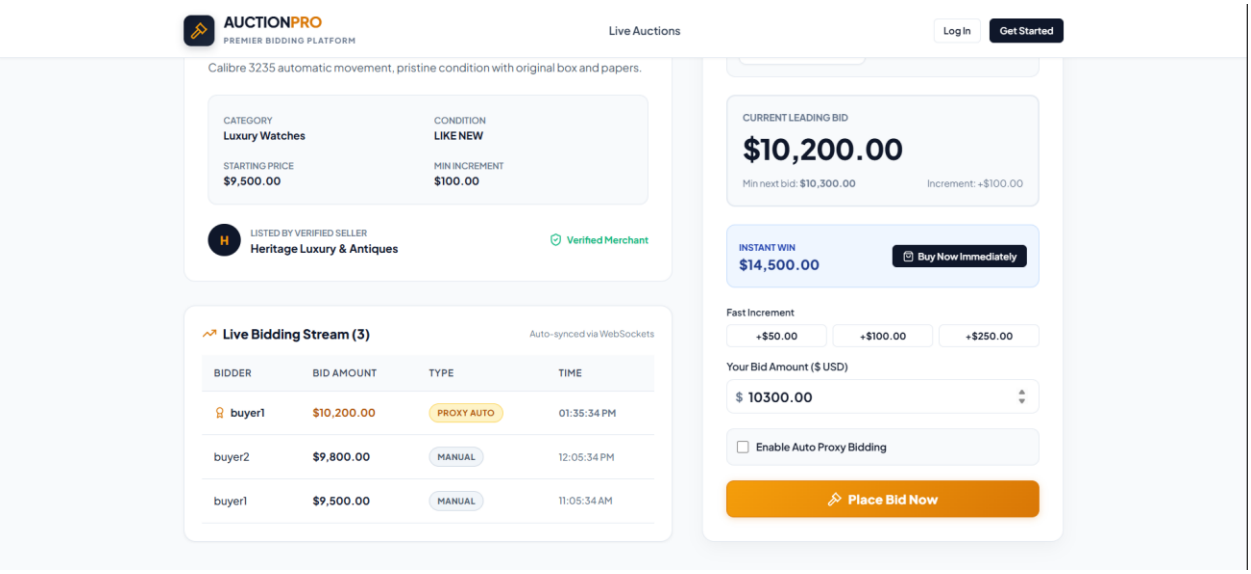
The following outputs describe the main screens and expected behavior of the application. Insert actual screenshots captured from the running application in the spaces below before final submission.

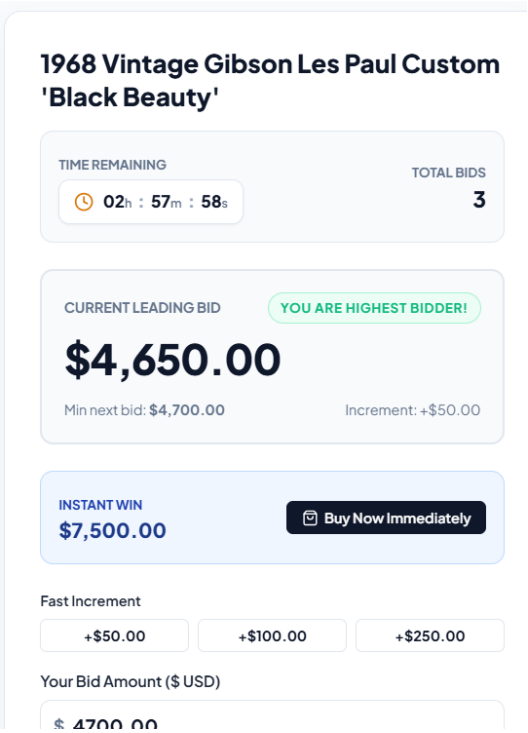


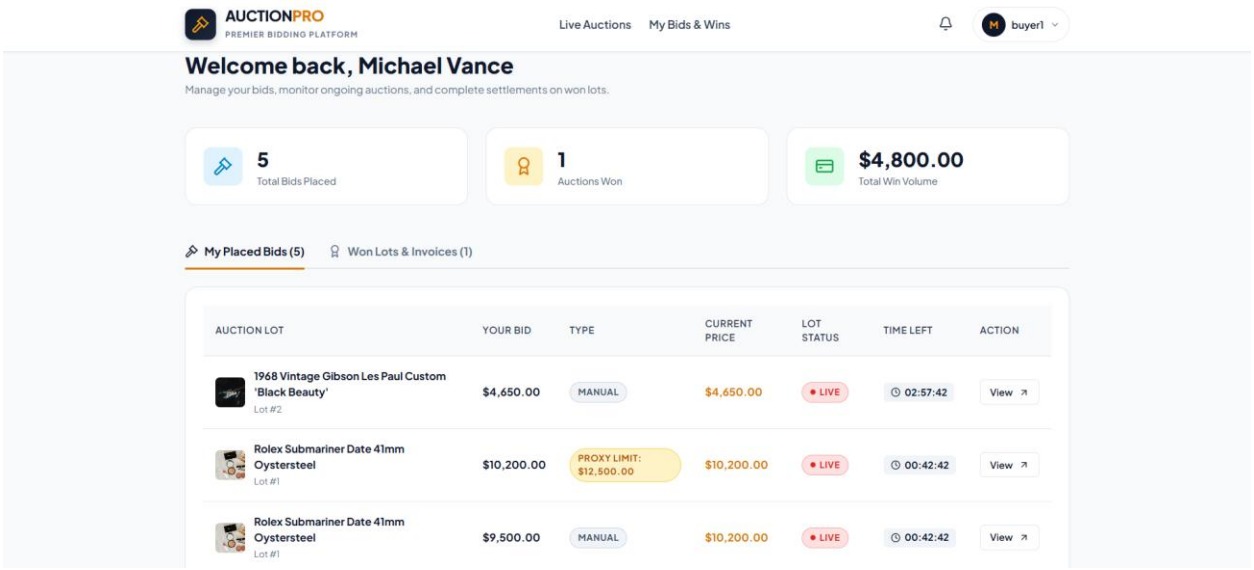


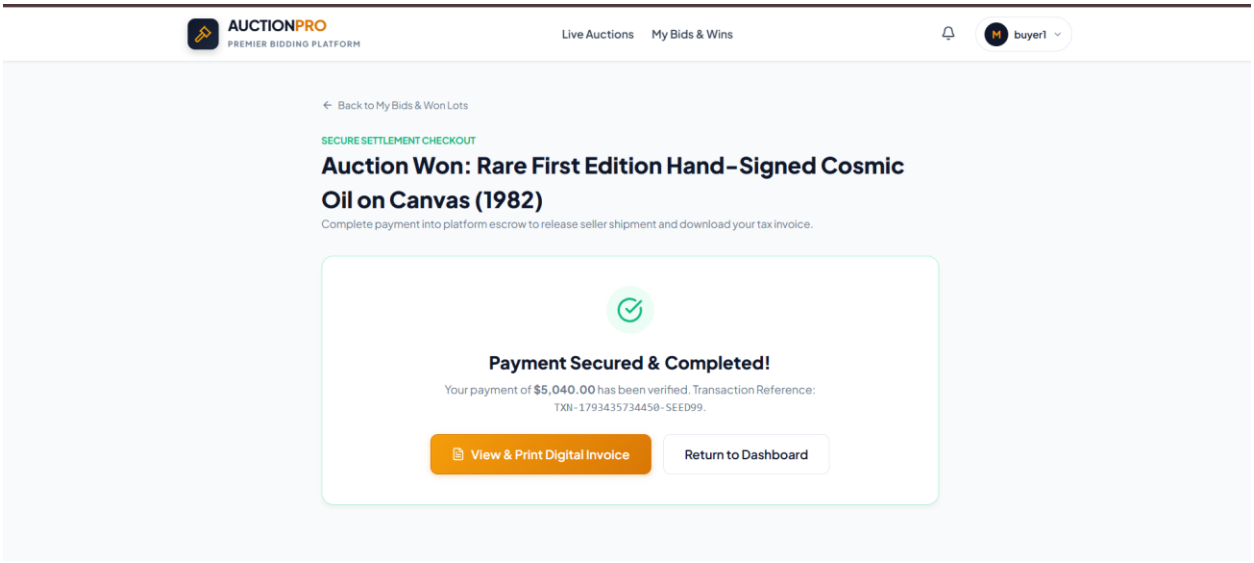

AUCTIONPRO
PREMIER BIDDING PLATFORM
Live Auctions
Log In
Get Started
Scale Tribute
Museum-grade hand-assembled 1:8 precision diecast sculpture with operational doors, leather upholstery...
CURRENT BID
1 bid
$1,350
Next: $1,375
11:59:09
Bid Now
Edition
Apple M2 and R1 dual-chip architecture with micro-OLED 23 million pixels 3D display system. Includes Du...
CURRENT BID
2 bids
$3,100
Next: $3,150
23:59:09
Bid Now
Colombian Diamond Ring
Certified Colombian emerald centerpiece accented by 1.20ct trapezoid side diamonds set in platinum 95...
CURRENT BID
0 bids
$7,800
Next: $7,900
1d 23:59:09
Bid Now
Real-Time Live Bidding
Millisecond WebSocket updates
Encrypted & Fair Escrow
Proxy bidding and automated settlement
Verified Authenticity
Certified sellers & digital invoices
AUCTIONPRO
The high-performance enterprise auction platform engineered with Java Spring Boot, MySQL, WebSockets and React.
DEMO CREDENTIALS
Admin: admin / admin123
Seller: seller1 / seller123
Buyer: buyer1 / buyer123
Categories
Luxury Watches
Art & Collectibles
Electronics & Tech
Fine Jewelry
Automotive
User Portals
Buyer Dashboard
Seller Portal
Schedule an Auction
Invoice Inquiries
Admin Console
Security & Platform
All bids are digitally signed and verified with server-side validation, proxy limits, and instant settlement.
Spring Security + JWT Enabled

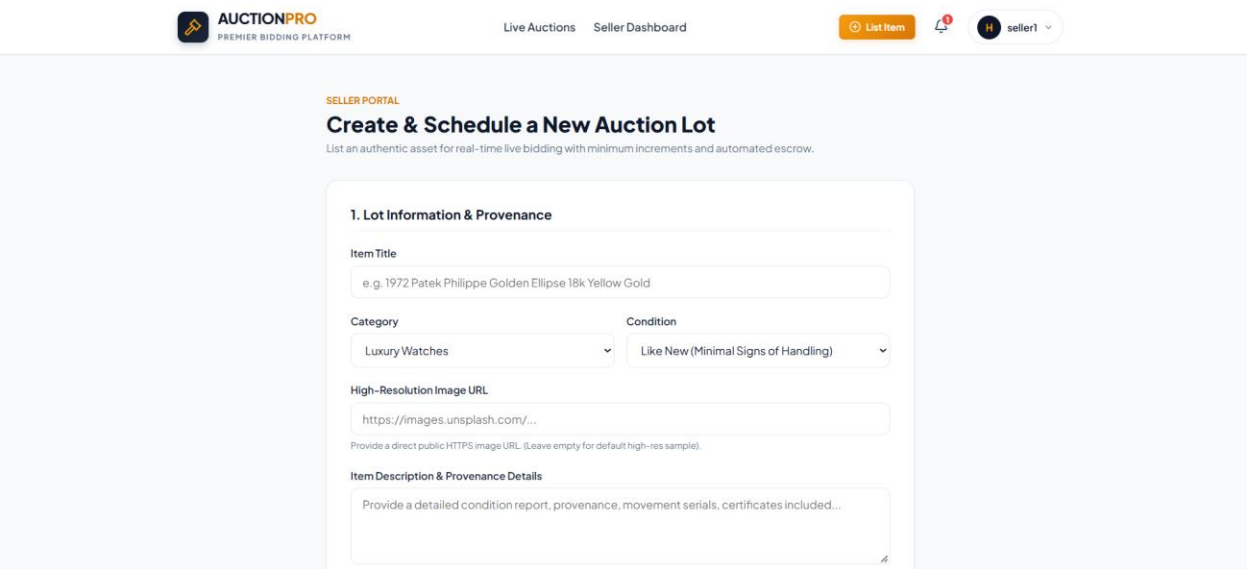

AUCTIONPRO
PREMIER BIDDING PLATFORM
Live Auctions
Log In
Get Started
LIVE NOW
Rolex Submariner Date 41mm Oystersteel
TIME REMAINING
00h : 43m : 50s
TOTAL BIDS
3
CURRENT LEADING BID
$10,200.00
Min next bid: $10,300.00
Increment: +$100.00
INSTANT WIN
$14,500.00
Buy Now Immediately
Fast Increment
+$50.00
+$100.00
+$250.00
Your Bid Amount ($ USD)
$ 10300.00
Lot Overview & Description
Cerachrom ceramic bezel in black, black dial with large luminescent hour markers. Calibre 3235 automatic movement, pristine condition with original box and papers.
CATEGORY
Luxury Watches
CONDITION
LIKE NEW
STARTING PRICE
MIN INCREMENT


AUCTIONPRO
PREMIER BIDDING PLATFORM
Live Auctions
Log In
Get Started
Calibre 3235 automatic movement, pristine condition with original box and papers.
CATEGORY
Luxury Watches
CONDITION
LIKE NEW
STARTING PRICE
$9,500.00
MIN INCREMENT
$100.00
LISTED BY VERIFIED SELLER
Heritage Luxury & Antiques
Verified Merchant
Live Bidding Stream (3)
Auto-synced via WebSockets
BIDDER
BID AMOUNT
TYPE
TIME
buyer1
$10,200.00
PROXY AUTO
01:35:34 PM
buyer2
$9,800.00
MANUAL
12:05:34 PM
buyer1
$9,500.00
MANUAL
11:05:34 AM
CURRENT LEADING BID
$10,200.00
Min next bid: $10,300.00
Increment: +$100.00
INSTANT WIN
$14,500.00
Buy Now Immediately
Fast Increment
+$50.00
+$100.00
+$250.00
Your Bid Amount ($ USD)
$ 10300.00
Enable Auto Proxy Bidding
Place Bid Now


1968 Vintage Gibson Les Paul Custom 'Black Beauty'
TIME REMAINING
02h : 57m : 58s
TOTAL BIDS
3
CURRENT LEADING BID
YOU ARE HIGHEST BIDDER!
$4,650.00
Min next bid: $4,700.00
Increment: +$50.00
INSTANT WIN
$7,500.00
Buy Now Immediately
Fast Increment
+$50.00
+$100.00
+$250.00
Your Bid Amount ($ USD)
$ 4700.00


AUCTIONPRO
PREMIER BIDDING PLATFORM
Live Auctions
My Bids & Wins
buyer1
Welcome back, Michael Vance
Manage your bids, monitor ongoing auctions, and complete settlements on won lots.
5
Total Bids Placed
1
Auctions Won
$4,800.00
Total Win Volume
My Placed Bids (5)
Won Lots & Invoices (1)
AUCTION LOT
YOUR BID
TYPE
CURRENT PRICE
LOT STATUS
TIME LEFT
ACTION
1968 Vintage Gibson Les Paul Custom 'Black Beauty'
Lot #2
$4,650.00
MANUAL
$4,650.00
LIVE
02:57:42
View
Rolex Submariner Date 41mm Oystersteel
Lot #1
$10,200.00
PROXY LIMIT: $12,500.00
$10,200.00
LIVE
00:42:42
View
Rolex Submariner Date 41mm Oystersteel
Lot #1
$9,500.00
MANUAL
$10,200.00
LIVE
00:42:42
View


AUCTIONPRO
PREMIER BIDDING PLATFORM
Live Auctions
My Bids & Wins
buyer1
Back to My Bids & Won Lots
SECURE SETTLEMENT CHECKOUT
Auction Won: Rare First Edition Hand-Signed Cosmic Oil on Canvas (1982)
Complete payment into platform escrow to release seller shipment and download your tax invoice.
Payment Secured & Completed!
Your payment of $5,040.00 has been verified. Transaction Reference: TXN-1793435734450-SEED99.
View & Print Digital Invoice
Return to Dashboard



- Bid Validation Flow: Bid request → Verify JWT and buyer role → Check auction is active → Compare bid with current price plus minimum increment → If invalid, reject with an error → If valid, persist bid atomically → Broadcast update to auction topic → Notify outbid participants.
- Database Relationship Overview: USER (1) → (many) PRODUCT; PRODUCT (1) → (many/associated) AUCTION as allowed by the application design; AUCTION (1) → (many) BID; AUCTION outcome → PAYMENT; PAYMENT → INVOICE. Foreign keys connect seller, buyer, auction and transaction records.
- Core data entities include User, Product, Auction, Bid, Payment, Invoice and Notification. Bid records retain the auction identifier, bidder identifier, amount and timestamp. Auction records retain schedule, minimum increment and status.
- Security and consistency: The backend is the authoritative source for bid acceptance. Protected actions require valid authentication and role authorization. Concurrent bid submissions should be tested to confirm that stored bid state remains consistent.
- Real-time communication: Connected clients subscribe to the topic for the relevant auction. The server publishes an update only after a bid has passed validation and has been persisted.
- Auction closure: The scheduled task checks configured end timestamps, changes the auction status to closed and prevents later bid submissions. The highest eligible bid is used to identify the winner according to the implemented auction rules.
- Payment workflow: A pending payment record is created after the auction outcome is recorded. Payment completion or failure is tracked separately from auction status.

- Invoice workflow: An invoice number and item/participant/amount details are associated with the completed auction. Any demo fee percentage must be treated as a configurable project value, not as a statutory tax conclusion.
- Notification workflow: In-app notifications are stored with read status. External email and SMS delivery is simulated when provider credentials are not configured.
- Testing should cover valid and invalid login, role restrictions, product creation, auction scheduling, minimum-increment checks, simultaneous bids, WebSocket updates, auction closure, winner identification and invoice generation.
- The actual application screenshots should be captured after the frontend and backend start successfully. The report intentionally does not include fabricated screenshots or unverified performance measurements.
- The implementation should be demonstrated using separate buyer, seller and administrator accounts. For live bidding, open the same auction in two browser sessions and verify that a bid in one session updates the other without a page refresh.
- The system's main contribution is the integration of auction scheduling, bid validation, real-time updates and post-auction records in one role-based application.

8. CONCLUSION & FUTURE SCOPE

The Auction Management Platform is designed to provide a centralized web application for product listing, auction scheduling, real-time bidding and post-auction activities. The proposed architecture combines React.js, Spring Boot, REST APIs, relational database persistence, Spring Security, JWT and WebSocket communication. Separate buyer, seller and administrator roles organize access to the system. Server-side bid validation supports minimum-increment enforcement, while WebSocket messaging provides live updates to connected participants. The planned modules also include proxy bidding, automatic auction closure, winner identification, payment-status tracking, invoice generation and notifications. The project establishes an integrated foundation for online auction management. Final claims about correctness, performance and security depend on the results of functional, integration, concurrency and security testing.

Future Scope: The platform can be enhanced by integrating a production payment gateway, real email/SMS providers, audit logs, dispute handling, seller verification and buyer reviews. Further work can improve fraud detection, accessibility, mobile usability and cloud deployment. Load and concurrency testing with documented test data can be used to measure update latency, bid consistency and behavior under increasing numbers of connected users. Database backup, monitoring, secure secret management and recovery from external service failures can also be added.

- Short-term: Complete end-to-end testing of registration, roles, listings, auctions, bidding, closure and invoices; capture actual output screenshots.
- Medium-term: Integrate a verified payment provider, real notification services, auction audit history and improved failure recovery.
- Long-term: Deploy on cloud infrastructure, add scalable multi-instance WebSocket support, perform load testing and introduce fraud/abuse detection

9. REFERENCES

1. B. Rumpe and G. Wimmel, “A Framework for Realtime Online Auctions,” arXiv:1409.6592, 2014. https://arxiv.org/abs/1409.6592

2. G. Cai, Y.-J. Chen and X. Gong, “Design of Online Auctions: Proxy versus Non-Proxy Settings,” Decision Support Systems, 52(2), 384–394, 2012. https://doi.org/10.1016/j.dss.2011.09.005

3. V. S. Borkar, M. S. Dave and R. K. Shyamasundar, “SeTiA: Secure Multi Auction System I: Auction Mechanisms,” Journal of Intelligent Systems, 14(1), 45–68, 2005. https://doi.org/10.1515/jisys.2005.14.1.45

4. V. S. Borkar, M. S. Dave and R. K. Shyamasundar, “SeTiA: Secure Multi Auction System II: Architecture and Implementation Issues,” Journal of Intelligent Systems, 14(1), 69–93, 2005. https://doi.org/10.1515/jisys.2005.14.1.69

5. eBay – Online Marketplace and Auction Listings. https://www.ebay.com/

6. Catawiki – Online Auctions. https://www.catawiki.com/

7. LiveAuctioneers – Online Auctions and Live Bidding. https://www.liveauctioneers.com/

8. Invaluable – Auction Catalogues and Results. https://www.invaluable.com/

9. Heritage Auctions – Online Auctions. https://www.ha.com/

10. Spring Boot Reference Documentation. https://docs.spring.io/spring-boot/index.html